॥वसुधैव कुटुम्बकम्॥

A DBMS PROJECT ON

DESIGN AND IMPLEMENTATION OF AN AIRLINE RESERVATION SYSTEM

SUBMITTED BY

Names:	**Paranjoy Bordoloi (24070122129)**
	Parmar Divyaansh (24070122130)
	Parth Sarathi Giri (24070122132)
Batch:	2024 - 2028
Faculty:	Prof. Jitendra Rajpurohit

Department of Computer Science
Symbiosis Institute of Technology, Pune

Contents

1 Introduction **3**
1.1 Contextual Overview of Aviation Data Management . . . 3
1.2 Evolution from Manual to Digital Frameworks . . . 3
1.3 Project Scope and Objectives . . . 3

2 Problem Statement **5**
2.1 The Crisis of Legacy and Manual Systems . . . 5
2.2 Specific Operational Pain Points . . . 5
2.3 The Imperative for a Relational Solution . . . 6

3 System Architecture and Modules **7**
3.1 Architectural Framework . . . 7
3.2 Functional Module Breakdown . . . 7
3.2.1 User Registration and Management Module . . . 7
3.2.2 Flight Search and Inquiry Module . . . 8
3.2.3 Booking and Reservation Module . . . 8
3.2.4 Payment and Transaction Module . . . 8
3.2.5 Administrator and Flight Management Module . . . 8
3.2.6 Cancellation and Notification Module . . . 9

4 Functional Requirements **10**
4.1 Passenger Functional Requirements . . . 10
4.2 Administrator Functional Requirements . . . 11
4.3 System/Non-Functional Requirements Impacting Database Design . . . 11

5 Entities, Attributes, and Relationships **12**
5.1 Entity Identification and Attribute Definition . . . 12
5.2 Relationship Analysis and Cardinalities . . . 14
5.3 Relationship Analysis Table . . . 15

6 Conceptual Design: ER and EER Diagrams **16**
6.1 ER Diagram Components . . . 16
6.2 Enhanced Entity-Relationship (EER) Modeling . . . 17
6.2.1 Person Generalization . . . 18
6.2.2 Employee Specialization . . . 19
6.2.3 Travel Class Specialization . . . 19

7 Relational Schema **20**
7.1 Table Definitions . . . 20
7.2 Integrity Constraints . . . 22

8 Functional Dependencies and Anomalies **23**
8.1 Functional Dependencies Identification . . . 23
8.2 Anomaly Analysis . . . 23

9 Application of Codd's Rules **25**

10 Normalization **27**
10.1 First Normal Form (1NF) 27
10.2 Second Normal Form (2NF) 27
10.3 Third Normal Form (3NF) 27
10.4 Boyce-Codd Normal Form (BCNF) 27

11 Conclusion **28**

1 Introduction

1.1 Contextual Overview of Aviation Data Management

The global aviation industry operates as a high-velocity network of logistics, customer service, and resource allocation, underpinned by complex information systems. At the core of this infrastructure lies the Airline Reservation System (ARS), a critical component of the broader Passenger Service System (PSS) [1]. In the contemporary digital economy, an ARS is not merely a mechanism for booking tickets; it is a sophisticated database management challenge that requires the seamless integration of inventory control, fare tariffs, passenger records, and flight schedules [2, 3]. The efficacy of an airline's operations—and by extension, its profitability—is directly correlated with the robustness of its underlying database architecture.

Historically, the management of airline reservations has evolved from labor-intensive manual processes to centralized mainframe environments, and finally to the distributed, web-enabled systems prevalent today [4]. Early systems were standalone entities, isolated from travel agents and other carriers, leading to inefficiencies and data silos. The advent of Computerized Reservation Systems (CRS) and Global Distribution Systems (GDS) transformed the landscape, allowing for real-time connectivity between airlines and distribution channels [5]. However, as airlines expand their routes and fleet sizes, legacy systems often struggle to maintain data consistency, concurrency, and scalability [6].

This project report details the design and theoretical simulation of a next-generation Airline Reservation System database. Adhering to the Phase 1 rubrics of the Database Management System (DBMS) project guidelines [7], the report provides an exhaustive analysis of the problem domain, functional requirements, conceptual modeling, and logical schema design. By leveraging a Relational Database Management System (RDBMS), the proposed solution aims to eliminate the anomalies inherent in manual processing, ensure data integrity through rigorous normalization, and support the complex transactional needs of modern air travel [8].

1.2 Evolution from Manual to Digital Frameworks

To understand the necessity of a robust DBMS solution, one must examine the limitations of the manual and early digital predecessors. In pre-digital eras, or within the context of small regional carriers operating with minimal technology, reservations were tracked using physical cards or flat-file spreadsheets [9]. This manual approach is fraught with peril: human error in data entry is rampant, retrieval of passenger history is sluggish, and the lack of real-time synchronization leads to significant operational risks such as overbooking or "lost" reservations [10].

The transition to digital systems was driven by the need for speed and accuracy. Modern systems must handle thousands of concurrent transactions—users searching for flights, booking seats, and cancelling itineraries simultaneously—without compromising data validity [11]. The shift from a flat-file approach to a relational model allows for the efficient storage of data across interconnected tables, reducing redundancy and improving retrieval times [12]. Furthermore, the integration of web technologies has shifted the power dynamic, empowering passengers to manage their own bookings directly, a functional requirement that demands a user-friendly and highly responsive database backend [13].

1.3 Project Scope and Objectives

The primary objective of this project is to architect a database solution that addresses the systemic failures of legacy reservation methods. The scope of this "Phase 1" report encompasses

the theoretical and logical design aspects of the system development life cycle.
Key Objectives:

- **Systematize Requirements:** To translate vague business needs into concrete functional requirements and functional dependencies [7].
- **Conceptual Modeling:** To visualize the domain using Entity-Relationship (ER) and Enhanced Entity-Relationship (EER) diagrams, capturing the nuances of entity interactions such as the relationship between aircraft, flights, and routes [14].
- **Logical Design:** To derive a normalized relational schema that adheres to Codd's rules, ensuring the database is free from update, insertion, and deletion anomalies [7, 15].
- **Data Integrity:** To define constraints that preserve the accuracy of data, such as enforcing valid passport numbers and preventing overlapping flight schedules for the same aircraft [16].

The subsequent sections of this report will meticulously deconstruct these objectives, providing a blueprint for a system capable of managing the lifecycle of a flight reservation from initial query to final landing.

2 Problem Statement

2.1 The Crisis of Legacy and Manual Systems

Despite the technological advancements in the broader industry, many airline sub-sectors and regional operators continue to rely on fragmented or poorly designed data management systems. The core problem addressed by this project is the inefficiency and operational risk introduced by these inadequate systems. A manual or flat-file based system is inherently incapable of handling the dynamic nature of modern aviation [17].

Operational Inefficiencies:
In a manual system, the process of booking a ticket involves a human operator physically checking a ledger or a standalone spreadsheet for seat availability. This introduces a critical latency; by the time the operator confirms availability, another agent might have already sold the seat [10]. This lack of concurrency control is a primary cause of double-booking, which results in revenue loss and severe customer dissatisfaction. Furthermore, manual systems make the retrieval of passenger records—essential for security checks and customer service—an arduous task involving physical searches through files [11].

Data Redundancy and Inconsistency:
Without a centralized RDBMS, data is often duplicated across different departments. The flight operations team might have one record of a flight's schedule, while the ticketing office has another. If a flight is delayed, this information often fails to propagate to the ticketing system in real-time, leaving passengers uninformed [18]. This inconsistency is a classic symptom of unnormalized data storage, where the same fact (e.g., flight departure time) is stored in multiple locations, leading to update anomalies.

2.2 Specific Operational Pain Points

The following specific challenges have been identified through domain analysis and serve as the drivers for the proposed system design:

1. **Inability to Select Seats:** A major deficiency in older systems is the lack of a visual seat map. Passengers are often assigned seats arbitrarily at the check-in counter, disregarding preferences for window or aisle seats [10]. This manual assignment slows down the check-in process and reduces passenger satisfaction.

2. **Notification Failures:** Legacy systems lack automated triggers. When a flight is cancelled or delayed due to weather, there is no automatic mechanism to query the `Passenger` database and dispatch notifications via SMS or email [13, 18].

3. **Complex Fare Calculation Errors:** Manual calculation of fares, which involves base rates, taxes, fuel surcharges, and dynamic class-based pricing, is prone to human arithmetic error. This can lead to financial leakage for the airline or overcharging of customers [2].

4. **Lack of Historical Insight:** Without a structured query language (SQL) capability, generating reports on route profitability, passenger demographics, or seasonal trends is virtually impossible [19]. Management lacks the data-driven insights necessary for strategic planning.

5. **Security and Access Control:** Flat-file systems do not offer granular access control. A ticketing agent should not have the same database privileges as a system administrator. The lack of role-based security in legacy systems poses a significant threat to data privacy [20].

2.3 The Imperative for a Relational Solution

The solution to these multifaceted problems lies in the implementation of a centralized Airline Reservation System built on a Relational Database Management System (RDBMS). An RDBMS solves the concurrency issue through transaction locking mechanisms (ACID properties), ensures data consistency through referential integrity constraints, and eliminates redundancy through normalization [12]. The proposed system will replace the fragmented manual processes with a unified platform where a change in one module (e.g., Flight Scheduling) instantly reflects in all others (e.g., Booking and Notifications).

3 System Architecture and Modules

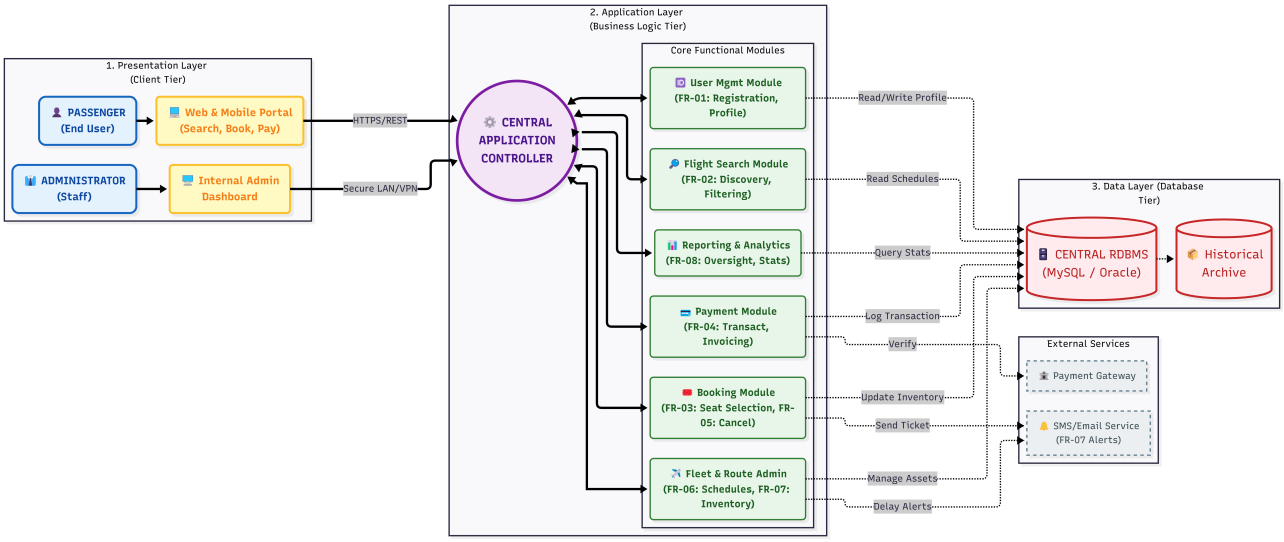

Figure 1: Three-Tier Architecture of the Airline Reservation System illustrating the interaction between Client, Application, and Data layers.

3.1 Architectural Framework

The proposed Airline Reservation System utilizes a multi-tier client-server architecture, designed to ensure scalability, security, and ease of maintenance [21, 22].

- **Presentation Layer (Client Tier):** This layer provides the interface for end-users (passengers) and internal users (administrators, staff). For passengers, this is a web-based portal accessible via browsers, facilitating flight search and booking [23]. For staff, a secure intranet portal provides access to administrative functions.
- **Application Layer (Business Logic Tier):** This middle tier hosts the core application logic. It processes requests from the client, enforces business rules (e.g., "a passenger cannot book a seat on a departed flight"), and manages sessions. It serves as the intermediary, preventing direct user access to the database [24].
- **Data Layer (Database Tier):** The foundation of the system is the RDBMS (e.g., MySQL or Oracle). This layer stores all persistent data, including flight inventories, passenger profiles, and transaction logs. It is responsible for data integrity, backup, and query execution [25].

3.2 Functional Module Breakdown

The system is decomposed into distinct functional modules, each addressing specific business requirements identified in the problem statement.

3.2.1 User Registration and Management Module

This module handles the lifecycle of user identities within the system. It distinguishes between distinct user roles: Passengers and Administrators [26].

- **Functionality:** Allows new customers to register by providing personal details (Name, Address, Passport Info) [2]. It manages secure login authentication via password hashing and session management [26].
- **Data Interaction:** Interacts primarily with the `Passenger` and `Login_Credentials` tables.

3.2.2 Flight Search and Inquiry Module

This is the primary interface for customer interaction. It allows users to query the database for flight availability based on specific criteria.

- **Functionality:** Enables searching by Origin, Destination, Date, and Class (Economy, Business). It retrieves flight schedules, durations, and pricing [9, 26].
- **Data Interaction:** Executes complex `SELECT` queries with `JOIN` operations across `Flight`, `Route`, `Airport`, and `Fare` tables.

3.2.3 Booking and Reservation Module

This core module manages the transactional aspect of selling seats. It is the most complex module as it involves concurrency control and inventory management [2].

- **Functionality:** Facilitates seat selection (visual map), passenger detail entry, and provisional booking creation. It checks the `AvailableSeats` attribute to prevent overbooking [27].
- **Data Interaction:** Performs `INSERT` operations into the `Booking` and `Ticket` tables and `UPDATE` operations on the `Flight` inventory.

3.2.4 Payment and Transaction Module

Responsible for the financial settlement of bookings.

- **Functionality:** Processes payments via integrated gateways, generates invoices, and updates the payment status of a booking from 'Pending' to 'Confirmed' [16, 28].
- **Data Interaction:** Manages the `Payment` and `Transaction` tables, ensuring a 1:1 link with the `Booking` table [29].

3.2.5 Administrator and Flight Management Module

A restricted-access module for airline operations staff to manage the supply side of the system.

- **Functionality:** Allows admins to Add, Modify, or Delete flight schedules and routes [26]. It also includes inventory control—assigning specific aircraft to routes based on capacity needs [30].
- **Data Interaction:** Direct manipulation of `Flight`, `Aircraft`, `Route`, and `Employee` tables.

3.2.6 Cancellation and Notification Module

Addresses the post-booking lifecycle and exception handling.

- **Functionality:** Allows passengers to cancel bookings, triggering a refund calculation and releasing the seat back to inventory [2]. It also monitors flight status changes (delays) and triggers notifications [10, 31].
- **Data Interaction:** Updates `Booking` status and `Flight` seat counts.

4 Functional Requirements

The functional requirements define the specific behaviors the system must exhibit to meet the business needs. These are categorized by the user roles interacting with the system.

4.1 Passenger Functional Requirements

FR-01: Registration and Profile Management

- The system shall allow unregistered users to create an account by providing a valid email address, password, full name, and phone number [26].
- The system shall allow registered users to view and update their profile information, including passport details and frequent flyer numbers [32].
- The system must validate that the password meets security complexity standards (e.g., minimum length, special characters) [2].

FR-02: Flight Search and Discovery

- The system shall provide a search interface where users enter Origin City, Destination City, and Travel Date [9, 32].
- The system shall display a list of available flights that match the criteria, showing Flight Number, Departure/Arrival Times, Duration, and Price [33].
- The system shall allow users to sort search results by Price (Low to High) or Duration (Shortest to Longest) [26].
- The system shall handle queries where no direct flight exists by suggesting connecting flights (if implemented) or displaying a "No Flights Available" message [9].

FR-03: Seat Selection and Booking

- The system shall display a graphical seat map for the selected flight, indicating occupied and available seats [32].
- The system shall allow the user to select a seat based on class (Economy, Business) [10].
- The system shall lock the selected seat for a predefined duration (e.g., 10 minutes) during the payment process to prevent double-booking [29].
- The system shall collect passenger details (Name, Age, Gender) for all travelers in the booking [9].

FR-04: Payment and Ticketing

- The system shall calculate the total fare, including base fare, taxes, and service charges [9].
- The system shall accept payment information and process the transaction securely [16].
- Upon successful payment, the system shall generate a unique Booking Reference Number (PNR) and a digital ticket/itinerary [32].

FR-05: Booking Management and Cancellation

- The system shall allow users to retrieve their booking using the PNR and Last Name [13].
- The system shall allow users to cancel a booking. The system must calculate the refund amount based on the cancellation policy and the time remaining before departure [26].
- The system shall allow users to print their boarding pass or ticket [10].

4.2 Administrator Functional Requirements

FR-06: Flight and Route Administration

- The system shall allow administrators to create new flight schedules, specifying the aircraft, route, and timing [26].
- The system shall allow administrators to modify existing flight details (e.g., change departure time) or cancel flights entirely [20].
- The system shall allow the definition of new Routes (Origin and Destination pairs) [14].

FR-07: Inventory and Fleet Management

- The system shall allow administrators to add new aircraft to the fleet, specifying the Model, Manufacturer, and Capacity [14].
- The system shall allow the assignment of crew members (Pilots, Cabin Crew) to specific flights [20].

FR-08: Reporting and Oversight

- The system shall provide a dashboard for administrators to view booking statistics, revenue reports, and flight occupancy rates [26].
- The system shall allow administrators to view the passenger manifest for any flight [30].

4.3 System/Non-Functional Requirements Impacting Database Design

FR-09: Data Integrity and Validation

- The system shall ensure that a flight cannot be scheduled if the assigned aircraft is already in use during that time block (Integrity Constraint) [16].
- The system shall ensure that the number of bookings for a flight does not exceed the aircraft's capacity [16].

FR-10: Scalability and Performance

- The database must support indexing on high-frequency search columns (e.g., `DepartureDate`, `Origin`, `Destination`) to ensure search results are returned in under 5 seconds [34].

5 Entities, Attributes, and Relationships

The Conceptual Design phase involves identifying the core entities that make up the airline domain and defining their attributes. This serves as the foundation for the ER diagram.

5.1 Entity Identification and Attribute Definition

Based on the research analysis [16, 35, 36, 37], the following entities have been identified.

1. **Passenger (Traveler)**
   - Represents the individual who utilizes the airline service.
   - **Attributes:**
     - `PassengerID` (Primary Key): A unique integer identifier for internal referencing.
     - `FirstName`: String.
     - `LastName`: String.
     - `PassportNumber`: String (Alphanumeric). Essential for international travel and identity verification. Must be unique.
     - `Email`: String. Used for sending tickets and notifications.
     - `Phone`: String. Contact number for SMS alerts.
     - `Address`: String. Billing or physical address.
     - `Age`: Integer. Used to categorize passengers (Adult, Child, Infant) for fare calculation.
2. **Flight (Scheduled Service)**
   - Represents a specific journey scheduled for a specific date and time.
   - **Attributes:**
     - `FlightID` (Primary Key): Unique identifier.
     - `FlightNumber`: String (e.g., "AI202"). Note that flight numbers are reused; `FlightID` distinguishes the specific instance on a specific date.
     - `DepartureTime`: DateTime.
     - `ArrivalTime`: DateTime.
     - `Status`: String/Enum (Scheduled, Delayed, Cancelled, Departed).
     - `AvailableSeats`: Integer. Tracks real-time inventory.
3. **Airport (Location)**
   - Represents the physical infrastructure where flights originate and terminate.
   - **Attributes:**
     - `AirportCode` (Primary Key): String (3 chars). Standard IATA code (e.g., LHR, DXB).
     - `Name`: String. Full name of the airport.
     - `City`: String.
     - `Country`: String.
     - `Timezone`: String. Crucial for calculating duration correctly across zones.
4. **Route (Path)**

- Represents the logical connection between two airports.
- **Attributes:**
  - `RouteID` (Primary Key): Unique identifier.
  - `Distance`: Integer. Used for fuel and fare calculations.
  - `EstimatedDuration`: Time. Standard flight time for this path.

5. **Aircraft (Physical Asset)**
   - Represents the specific plane assigned to a flight.
   - **Attributes:**
     - `AircraftID` (Primary Key): Unique identifier.
     - `RegistrationNumber`: String. Tail number (e.g., N12345).
     - `Model`: String (e.g., "Boeing 737").
     - `Manufacturer`: String.
     - `TotalCapacity`: Integer.
     - `BusinessSeats`: Integer.
     - `EconomySeats`: Integer.
6. **Booking (Reservation)**
   - Represents the contract between the passenger and the airline.
   - **Attributes:**
     - `BookingID` (Primary Key): Unique PNR.
     - `BookingDate`: DateTime.
     - `TravelClass`: String (Economy, Business).
     - `SeatNumber`: String (e.g., "12A").
     - `Status`: String (Confirmed, Cancelled, Pending).
     - `TotalAmount`: Decimal.
7. **Payment (Financial Transaction)**
   - Represents the exchange of funds for the booking.
   - **Attributes:**
     - `PaymentID` (Primary Key): Unique identifier.
     - `Amount`: Decimal.
     - `PaymentMethod`: String (Credit Card, PayPal).
     - `TransactionDate`: DateTime.
     - `TransactionReference`: String. Gateway confirmation code.
8. **Employee (Staff)**
   - Represents airline personnel involved in operations.
   - **Attributes:**
     - `EmployeeID` (Primary Key).
     - `Name`: String.
     - `Role`: String (Pilot, Crew, Admin).
     - `DateHired`: Date.

5.2 Relationship Analysis and Cardinalities

The interaction between these entities defines the business logic of the system [14, 38].

- **Passenger - Booking:**
  - **Relationship:** A Passenger *Makes* a Booking.
  - **Cardinality:** One-to-Many (1:N). A passenger can make multiple bookings over time. A specific booking record is associated with one primary passenger.
- **Flight - Booking:**
  - **Relationship:** A Booking *Is_For* a Flight.
  - **Cardinality:** One-to-Many (1:N). One flight holds many bookings. A single booking (ticket) is for one specific flight leg.
- **Flight - Aircraft:**
  - **Relationship:** A Flight *Uses* an Aircraft.
  - **Cardinality:** Many-to-One (N:1). An aircraft flies many flights over its lifetime. A specific flight instance uses exactly one aircraft.
- **Flight - Route:**
  - **Relationship:** A Flight *Follows* a Route.
  - **Cardinality:** Many-to-One (N:1). Multiple flight instances (e.g., the daily 9 AM flight) follow the same defined route.
- **Route - Airport:**
  - **Relationship:** A Route *Connects* Airports.
  - **Cardinality:** Binary relationship linking an Origin Airport and a Destination Airport.
- **Booking - Payment:**
  - **Relationship:** A Booking *Generates* a Payment.
  - **Cardinality:** One-to-One (1:1). For this system design, we assume one booking has one associated payment transaction [29].
- **Employee - Flight:**
  - **Relationship:** Employees *Operate* a Flight.
  - **Cardinality:** Many-to-Many (M:N). A flight requires a crew (multiple employees). An employee works on multiple flights. This requires an associative entity `Crew_Assignment` [39].

5.3 Relationship Analysis Table

The interaction between these entities defines the business logic of the system. The cardinalities and relationships are summarized below.

Entity A	**Relationship**	**Entity B**	**Cardinality**	**Description**
Passenger	Makes	Booking	1:N	A passenger can make multiple bookings; a booking belongs to one passenger.
Booking	Is_For	Flight	N:1	Multiple bookings exist for a single flight; a ticket is for one specific flight.
Flight	Uses	Aircraft	N:1	An aircraft is reused for many flights; a scheduled flight uses one aircraft.
Flight	Follows	Route	N:1	Many flights follow the same path; a specific flight schedule follows one route.
Route	Connects	Airport	M:N	A route connects two airports (Origin and Destination).
Booking	Generates	Payment	1:1	Each confirmed booking has exactly one payment record.
Employee	Operates	Flight	M:N	Multiple crew operate a flight; employees work on multiple flights.

Table 1: Relationship Analysis and Cardinalities

6 Conceptual Design: ER and EER Diagrams

The Entity-Relationship (ER) model provides a graphical representation of the logical structure of the database. This section describes the components and the advanced modeling concepts used.

6.1 ER Diagram Components

The ER diagram for the Airline Reservation System is constructed using standard Crow's Foot notation concepts.

- **Strong Entities:** `Passenger`, `Flight`, `Airport`, `Aircraft`, `Airline`, `Route`. These entities have their own primary keys and exist independently.
- **Weak Entities:** `Seat` can be modeled as a weak entity dependent on `Aircraft` (identified by `SeatNumber` + `AircraftID`), though often it is simplified as an attribute in smaller designs. In this comprehensive report, `Ticket` is often considered a weak entity dependent on `Booking` if modeled separately [40].
- **Associative Entities:** `Crew_Assignment` resolves the many-to-many relationship between `Employee` and `Flight`. It contains attributes like `Role_In_Flight` (e.g., Pilot, Co-Pilot, Head Steward) [41].

Diagram Narrative: The central hub is the `Flight` entity. It has outgoing relationships to `Airport` (Origin and Destination), `Aircraft` (the physical plane), and `Route` (the path). The `Booking` entity acts as the transactional link, connecting `Passenger` to `Flight`. `Payment` is an appendage to `Booking`, strictly 1:1.

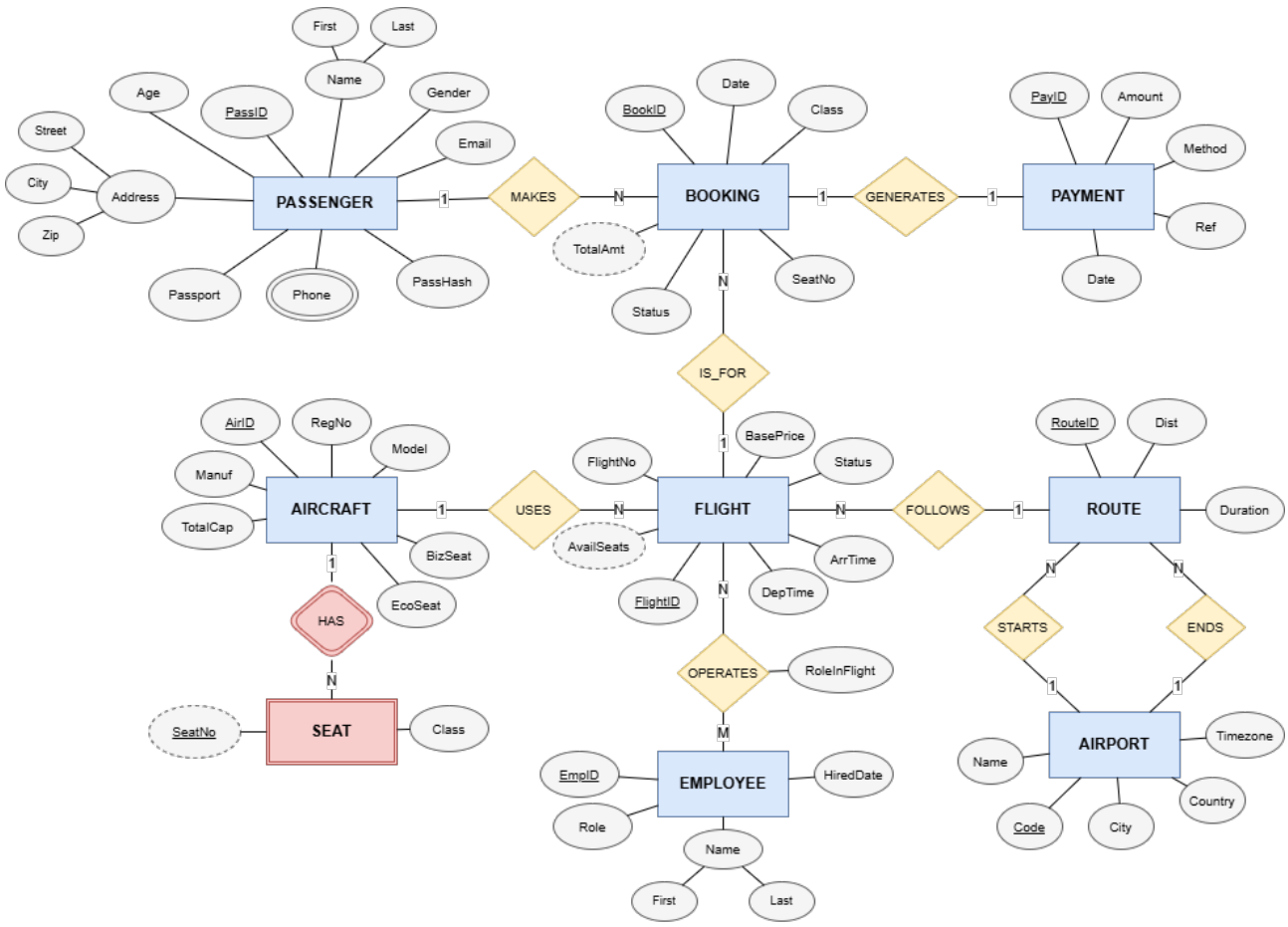

Figure 2: Entity-Relationship (ER) Diagram for Airline Reservation System

6.2 Enhanced Entity-Relationship (EER) Modeling

To capture the complexity of the domain effectively, Enhanced ER concepts such as **Generalization** (Superclass/Subclass) and **Specialization** are employed [42, 43].

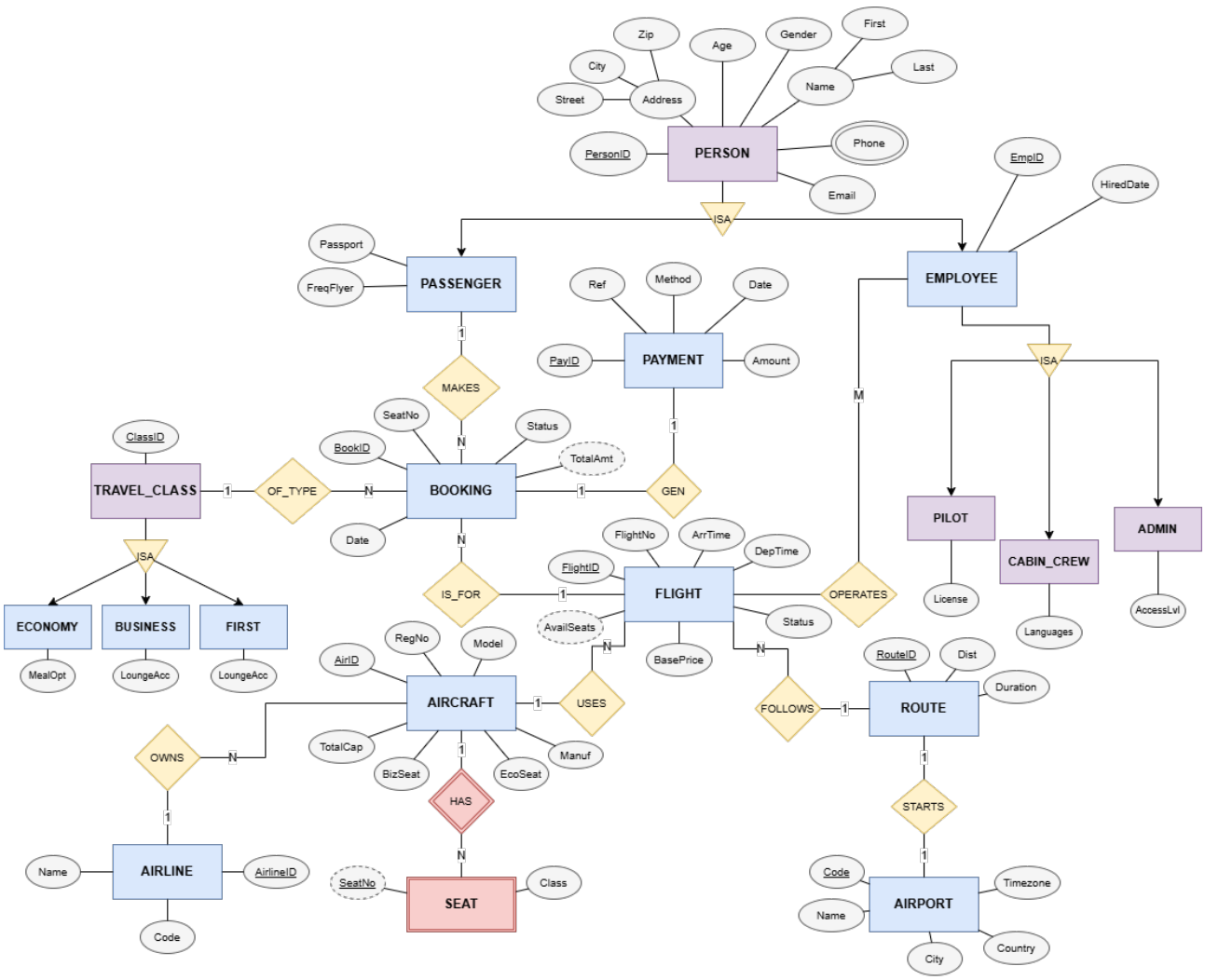

Figure 3: Enhanced Entity-Relationship (EER) Diagram

6.2.1 Person Generalization

There is a significant overlap in attributes between `Passenger` and `Employee` (Name, Address, Contact Info).

- **Superclass:** `Person`
  - Attributes: `PersonID`, `Name`, `Address`, `PhoneNumber`, `Email`.
- **Subclasses:**
  - `Passenger`: Adds `PassportNumber`, `FrequentFlyerMiles`.
  - `Employee`: Adds `EmployeeID`, `Salary`, `Department`.
- **Constraint:** This is an **Overlapping** specialization (a person could theoretically be both an employee and a passenger) and **Partial** participation (a person might exist in the system as a contact reference without being either active role) [41].

6.2.2 Employee Specialization

Within the `Employee` entity, distinct roles require different data points [39].

- **Superclass:** `Employee`
- **Subclasses:**
  - `Pilot`: Attributes include `LicenseNumber`, `FlightHoursLog`, `MedicalCertExpiry`.
  - `CabinCrew`: Attributes include `LanguageSkills`.
  - `AdminStaff`: Attributes include `AccessLevel`, `SystemPrivileges`.
- **Constraint: Disjoint** (an employee typically performs one primary role at a time) and **Total** participation (every employee must be assigned a role).

6.2.3 Travel Class Specialization

- **Superclass:** `TravelClass`
- **Subclasses:** `Economy`, `Business`, `First`.
- This allows the system to store class-specific attributes, such as `LoungeAccess` (Boolean) for Business/First, or `MealSelection` options [44].

7 Relational Schema

The conceptual ER/EER model is transformed into a logical Relational Schema. This involves mapping entities to tables, attributes to columns, and relationships to foreign keys.

7.1 Table Definitions

Table 2: Table: Airport

Attribute	**Data Type**	**Constraints**	**Description**
AirportCode	VARCHAR(3)	PK	Standard IATA code (e.g., LHR).
Name	VARCHAR(100)	NOT NULL	Full name of the airport.
City	VARCHAR(50)	NOT NULL	Location city.
Country	VARCHAR(50)	NOT NULL	Location country.

Schema: `Airport (AirportCode, Name, City, Country)`

Table 3: Table: Airline

Attribute	**Data Type**	**Constraints**	**Description**
AirlineID	INT	PK	Unique identifier.
Name	VARCHAR(100)	NOT NULL	Name of the airline.
Code	VARCHAR(3)	Unique	ICAO/IATA Airline code.

Schema: `Airline (AirlineID, Name, Code)`

Table 4: Table: Aircraft

Attribute	**Data Type**	**Constraints**	**Description**
AircraftID	INT	PK	Unique identifier.
RegistrationNumber	VARCHAR(20)	Unique	Tail number (e.g., N12345).
Model	VARCHAR(50)	NOT NULL	Aircraft model.
Capacity	INT	NOT NULL	Total seating capacity.
AirlineID	INT	FK	References Airline.

Schema: `Aircraft (AircraftID, RegistrationNumber, Model, Capacity, AirlineID)`

Table 5: Table: Route

Attribute	**Data Type**	**Constraints**	**Description**
RouteID	INT	PK	Unique identifier.
OriginCode	VARCHAR(3)	FK	References Airport (Origin).
DestCode	VARCHAR(3)	FK	References Airport (Dest).
Distance	INT		Distance in Km/Miles.

Schema: `Route (RouteID, OriginCode, DestCode, Distance)`

Table 6: Table: Flight

Attribute	Data Type	Constraints	Description
FlightID	INT	PK	Unique identifier.
FlightNumber	VARCHAR(10)	NOT NULL	Commercial flight no.
RouteID	INT	FK	References Route.
AircraftID	INT	FK	References Aircraft.
DepartureTime	DATETIME	NOT NULL	Scheduled departure.
ArrivalTime	DATETIME	NOT NULL	Scheduled arrival.
BasePrice	DECIMAL	NOT NULL	Standard ticket cost.
Status	VARCHAR(20)	Check	Scheduled/Delayed/Cancelled.

Schema: `Flight (FlightID, FlightNumber, RouteID, AircraftID, DepartureTime, ArrivalTime BasePrice, Status)`

Table 7: Table: Passenger

Attribute	Data Type	Constraints	Description
PassengerID	INT	PK	Unique identifier.
FirstName	VARCHAR(50)	NOT NULL	First Name.
LastName	VARCHAR(50)	NOT NULL	Last Name.
Email	VARCHAR(100)	Unique	Contact Email.
PassportNumber	VARCHAR(20)	Unique	Travel document ID.
Phone	VARCHAR(15)		Contact number.

Schema: `Passenger (PassengerID, FirstName, LastName, Email, PassportNumber, Phone)`

Table 8: Table: Booking

Attribute	Data Type	Constraints	Description
BookingID	INT	PK	Unique PNR.
PassengerID	INT	FK	References Passenger.
FlightID	INT	FK	References Flight.
BookingDate	DATETIME	NOT NULL	Date of reservation.
SeatNumber	VARCHAR(5)		Assigned seat.
ClassType	VARCHAR(20)		Economy/Business.
Status	VARCHAR(20)		Confirmed/Pending.

Schema: `Booking (BookingID, PassengerID, FlightID, BookingDate, SeatNumber, ClassType, Status)`

Table 9: Table: Payment

Attribute	Data Type	Constraints	Description
PaymentID	INT	PK	Unique identifier.
BookingID	INT	FK, Unique	References Booking.
Amount	DECIMAL	NOT NULL	Amount paid.
PaymentMethod	VARCHAR(20)		Card/UPI/Cash.
TransactionDate	DATETIME		Date of payment.

Schema: `Payment (PaymentID, BookingID, Amount, PaymentMethod, TransactionDate)`

Table 10: Table: Crew_Assignment

Attribute	**Data Type**	**Constraints**	**Description**
AssignmentID	INT	PK	Unique identifier.
EmployeeID	INT	FK	References Employee.
FlightID	INT	FK	References Flight.
Role	VARCHAR(20)		Pilot/Co-Pilot/Crew.

Schema: `Crew_Assignment (AssignmentID, EmployeeID, FlightID, Role)`

7.2 Integrity Constraints

- **Entity Integrity:** All Primary Keys must be non-null and unique.
- **Referential Integrity:** All Foreign Keys must match a valid Primary Key in the parent table. For example, a `Booking` cannot reference a `PassengerID` that does not exist.
- **Domain Integrity:** Data types are strictly enforced (e.g., `DepartureTime` must be a valid DATETIME).
- **Business Rules (Triggers/Check Constraints):**
  - `DepartureTime` must be earlier than `ArrivalTime`.
  - `SeatNumber` must not be duplicated for the same `FlightID` (Unique Constraint on composite `FlightID` + `SeatNumber`).

8 Functional Dependencies and Anomalies

A critical aspect of database design is ensuring the schema is efficient and free from logical errors. This is achieved by analyzing Functional Dependencies (FDs) and identifying potential anomalies [47].

8.1 Functional Dependencies Identification

Functional dependency describes the relationship between attributes. We denote this as $X \rightarrow Y$, meaning X uniquely determines Y.

Analysis of the `Flight` Table:

- $FlightID \rightarrow FlightNumber, RouteID, AircraftID, DepartureTime, ArrivalTime$. (The Primary Key determines all other attributes).
- If we had stored `OriginCity` in the `Flight` table, we would have: $RouteID \rightarrow OriginCode \rightarrow OriginCity$. This represents a **Transitive Dependency**, which is undesirable in 3NF.

Analysis of the `Booking` Table:

- $BookingID \rightarrow PassengerID, FlightID, BookingDate, SeatNumber$.
- The combination $(FlightID, SeatNumber) \rightarrow BookingID$. (A specific seat on a specific flight identifies the booking).

Analysis of the `Aircraft` Table:

- $AircraftID \rightarrow Model, Capacity$.
- $Model \rightarrow Manufacturer, TotalCapacity$. (Note: All aircraft of the "Boeing 737" model generally have the same base specifications, though configurations vary. If specific configuration depends on the individual plane, then `AircraftID` determines Capacity).

8.2 Anomaly Analysis

Anomalies are problems that occur in poorly structured databases when performing data manipulation operations.

1. **Insertion Anomaly:**
   - **Problem:** Suppose we want to add a new Aircraft model (e.g., "Airbus A350") to our database to signify future acquisition, but we haven't bought a specific plane yet. If `Model` information is stored only in the `Aircraft` table which requires an `AircraftID` (specific plane), we cannot store the model data without creating a fake plane record.
   - **Solution:** Create a separate `Aircraft_Type` reference table.
2. **Deletion Anomaly:**
   - **Problem:** Consider a table `Flight_Info` that stores both flight schedule and pilot details: $(FlightID, PilotName, PilotLicense)$. If we cancel the flight (delete the row), we inadvertently lose the information about the pilot (Name and License) from the system.
   - **Solution:** Separate `Employee` data into its own table, linked by `Crew_Assignment`.

3. **Update Anomaly:**
   - **Problem:** If `PassengerAddress` is stored in the `Booking` table, and a passenger moves house, we must update every single past and future booking record for that passenger. Failing to update one row leaves the database in an inconsistent state.
   - **Solution:** Store `Address` only in the `Passenger` table. `Booking` references `PassengerID`.

9 Application of Codd's Rules

The proposed Airline Reservation System adheres to Dr. E.F. Codd's 12 rules for Relational Database Management Systems. These rules ensure the database is truly relational, consistent, and independent of the physical implementation.

Rule 0: The Foundation Rule
The system must be able to manage its database entirely through its relational capabilities. *Application:* Our ARS uses a standard RDBMS (like MySQL) to handle all data definition, manipulation, and control operations.

Rule 1: The Information Rule
All information in the database is represented in one and only one way: by values in column positions within rows of tables. *Application:* All data, including flight schedules, passenger lists, and booking statuses, are stored strictly in the tables defined in the Relational Schema (Section 7). No data is hidden in pointers or file headers.

Rule 2: The Guaranteed Access Rule
Each and every datum (atomic value) is guaranteed to be logically accessible by resorting to a combination of table name, primary key value, and column name. *Application:* Any specific data point, such as a passenger's email, can be retrieved unambiguously using: `SELECT Email FROM Passenger WHERE PassengerID = 101`.

Rule 3: Systematic Treatment of Null Values
Null values (distinct from zero or empty strings) are supported for representing missing or inapplicable information. *Application:* The `Phone` attribute in the `Passenger` table allows NULL values if a user does not provide a secondary contact, distinguishing it from an empty string.

Rule 4: Dynamic Online Catalog Based on the Relational Model
The database description is represented at the logical level in the same way as ordinary data. *Application:* The metadata (schema definitions) for `Flight`, `Booking`, etc., are stored in system tables (e.g., `information_schema`) and can be queried using standard SQL.

Rule 5: The Comprehensive Data Sublanguage Rule
The system must support at least one language that is comprehensive (supporting data definition, view definition, manipulation, integrity constraints, and transaction management). *Application:* The system utilizes SQL (Structured Query Language) for all CRUD operations, table creation, and transaction control (COMMIT/ROLLBACK).

Rule 6: The View Updating Rule
All views that are theoretically updatable are also updatable by the system. *Application:* A view creating a virtual table of "Today's Flights" can be used by admins to update the status of a flight, which reflects in the base `Flight` table.

Rule 7: High-level Insert, Update, and Delete
The capability of handling a base relation or a derived relation as a single operand applies not only to the retrieval of data but also to the insertion, update, and deletion of data. *Application:* The system allows bulk updates, such as cancelling all bookings for a specific flight using a single SQL command: `UPDATE Booking SET Status='Cancelled' WHERE FlightID=202`.

Rule 8: Physical Data Independence
Application programs and terminal activities remain logically unimpaired whenever any changes are made in either storage representations or access methods. *Application:* If the underlying storage of the `Booking` table is moved from a HDD to an SSD, or if the indexing method changes from B-Tree to Hash, the frontend application code remains unchanged.

Rule 9: Logical Data Independence
Application programs and terminal activities remain logically unimpaired when information-preserving changes of any kind that theoretically permit unimpairment are made to the base tables. *Application:* If we add a new column `MealPreference` to the `Booking` table, existing queries that select `BookingID` and `SeatNumber` will continue to function without error.

Rule 10: Integrity Independence
Integrity constraints specific to a particular relational database must be definable in the relational data sublanguage and storable in the catalog, not in the application programs. *Application:* Constraints like Primary Keys (e.g., `PassengerID`) and Check Constraints (e.g., `DepartureTime < ArrivalTime`) are defined within the SQL schema, not in the application code.

Rule 11: Distribution Independence
The distribution of data across various locations should be transparent to the user. *Application:* If the database is scaled (sharded) so that European flights are stored on a server in London and Asian flights in Singapore, the user executes the same SQL query regardless of where the data resides.

Rule 12: The Non-Subversion Rule
If the system has a low-level (record-at-a-time) interface, that interface cannot be used to subvert the system, bypassing security or integrity constraints. *Application:* A system administrator cannot use a backend tool to bypass the Foreign Key constraint on `Booking` to create a reservation for a non-existent flight.

10 Normalization

Normalization is the process of organizing data to minimize redundancy and dependency. We apply Codd's rules to refine the schema [7, 48].

10.1 First Normal Form (1NF)

Rule: atomic values, unique column names, no repeating groups.

- **Application:** In the conceptual phase, `Passenger` might have had a multi-valued attribute `PhoneNumbers` (Mobile, Home). In 1NF, we cannot store "123-456, 987-654" in a single cell.
- **Action:** We separate this into a `Passenger_Phone` table: $(PassengerID, PhoneType, Number)$. Now all tables contain only atomic values.

10.2 Second Normal Form (2NF)

Rule: In 1NF and no partial dependencies (attributes must depend on the *entire* primary key).

- **Application:** Consider the `Crew_Assignment` table with Composite PK $(EmployeeID, FlightID)$. If we included `EmployeeName` in this table, it would depend only on `EmployeeID` (part of the key), not `FlightID`. This is a partial dependency.
- **Action:** `EmployeeName` is moved to the `Employee` table. `Crew_Assignment` retains only `Role`, which depends on *both* the employee and the flight (e.g., Jane is the Pilot for Flight A, but Co-Pilot for Flight B).

10.3 Third Normal Form (3NF)

Rule: In 2NF and no transitive dependencies (non-key attributes depend only on the primary key).

- **Application:** In the `Flight` table, if we stored `OriginCity`, the dependency chain is $FlightID \rightarrow RouteID \rightarrow OriginCode \rightarrow OriginCity$. `OriginCity` is transitively dependent on `FlightID`.
- **Action:** We place `City` in the `Airport` table. `Flight` references `RouteID`, and `Route` references `AirportCode`. The `Flight` table now only contains data directly related to the specific flight instance.

10.4 Boyce-Codd Normal Form (BCNF)

Rule: Every determinant is a candidate key.

- **Application:** In the `Booking` table, `BookingID` determines `SeatNumber`. Also, $(FlightID, SeatNumber)$ determines `BookingID`. Since both determinants (`BookingID` and `FlightID`+`SeatNumber`) are candidate keys, the table satisfies BCNF.

11 Conclusion

This project report has presented a comprehensive design for an Airline Reservation System, adhering to the strict academic and technical standards of the Phase 1 DBMS project rubrics. By systematically analyzing the inefficiencies of legacy systems—ranging from data redundancy to lack of concurrency control—the report established a clear imperative for a relational database solution.

The **Functional Requirements** provided a detailed roadmap for the system's capabilities, ensuring that the needs of both passengers (seat selection, real-time booking) and administrators (inventory control, route management) are met. The **System Architecture** section outlined a robust modular design, ensuring scalability and security.

The core of the report, the **Conceptual and Logical Design**, utilized advanced ER/EER modeling techniques to accurately represent the aviation domain, capturing complex relationships like crew assignments and aircraft scheduling. Finally, the **Normalization** process rigorously refined the schema to 3NF/BCNF, guaranteeing a database structure that is free from anomalies and optimized for data integrity. This design serves as a technically sound blueprint for the subsequent implementation phases of SQL coding and application development.

References

[1] IATA, "Passenger Service Systems (PSS) Explained," https://www.iata.org/en/programs/passenger/pss/.

[2] T. H. Davenport, *Competing on Analytics: The New Science of Winning*. Harvard Business Review Press, 2007.

[3] B. Smith, J. F. Leimkuhler, and R. M. Darrow, "Yield Management at American Airlines," *Interfaces*, vol. 22, no. 1, pp. 8–31, 1992.

[4] Britannica, "History of Airline Reservation Systems," https://www.britannica.com/technology/computerized-reservation-system.

[5] D. G. Copeland, R. O. Mason, and J. L. McKenney, *Sabre: The Development of an Information System*. Harvard Business School Case Study, 1995.

[6] P. Belobaba, A. Odoni, and C. Barnhart, "The Global Airline Industry," *Aerospace Series*, 2009.

[7] Symbiosis Institute of Technology, *DBMS Project Guidelines*, 2026.

[8] R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[9] AltexSoft, "Evolution of Airline Reservation Systems," https://www.altexsoft.com/blog/travel/airline-reservation-systems-history/.

[10] W. E. O'Connor, "An Introduction to Airline Economics," *Praeger*, 2001.

[11] M. Stonehouse and J. Snow, "The Future of Airline Distribution," *Journal of Revenue and Pricing Management*, 2004.

[12] C. J. Date, *An Introduction to Database Systems*, 8th ed. Addison-Wesley, 2003.

[13] J. Wirtz, P. Chew, and C. Lovelock, "Services Marketing: People, Technology, Strategy," *World Scientific*, 2012.

[14] P. Chen, *The Entity-Relationship Model: Toward a Unified View of Data*. ACM Transactions on Database Systems, 1976.

[15] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[16] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 6th ed. McGraw-Hill, 2010.

[17] "Legacy Systems in Aviation," https://www.w3.org/.

[18] K. Laudon and J. Laudon, *Management Information Systems: Managing the Digital Firm*. Pearson, 2012.

[19] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*. Pearson, 2014.

[20] M. Whitman and H. Mattord, *Principles of Information Security*. Cengage Learning, 2011.

[21] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley, 2003.

[22] Techopedia, "Three-Tier Architecture Definition," https://www.techopedia.com/definition/24649/three-tier-architecture.

[23] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," *University of California, Irvine*, 2000.

[24] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[25] C. Coronel and S. Morris, *Database Systems: Design, Implementation, & Management*. Cengage Learning, 2016.

[26] I. Sommerville, *Software Engineering*. Pearson, 2011.

[27] P. A. Bernstein and E. Newcomer, *Principles of Transaction Processing*. Morgan Kaufmann, 2009.

[28] "Payment Card Industry Data Security Standard (PCI DSS)," https://www.pcisecuritystandards.org/.

[29] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992.

[30] S. Chopra and P. Meindl, *Supply Chain Management: Strategy, Planning, and Operation*. Pearson, 2013.

[31] G. Coulouris, J. Dollimore, and T. Kindberg, *Distributed Systems: Concepts and Design*. Addison-Wesley, 2011.

[32] B. Shneiderman and C. Plaisant, *Designing the User Interface*. Pearson, 2010.

[33] IATA, "Standard Schedules Information Manual," https://www.iata.org/.

[34] M. Stonebraker, "The Case for Shared Nothing," *IEEE Database Engineering Bulletin*, 1986.

[35] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.

[36] OpenFlights, "Airline Codes Dataset," https://openflights.org/data.html.

[37] D. Norman, *The Design of Everyday Things*. Basic Books, 2013.

[38] T. Teorey, *Database Modeling and Design*. Morgan Kaufmann, 1999.

[39] J. Rumbaugh, I. Jacobson, and G. Booch, *The Unified Modeling Language Reference Manual*. Addison-Wesley, 2004.

[40] S. Atzeni and V. De Antonellis, *Relational Database Theory*. Benjamin-Cummings, 1993.

[41] R. Barker, *CASE*Method: Entity Relationship Modelling*. Addison-Wesley, 1990.

[42] G. Wiederhold, *Database Design*. McGraw-Hill, 1983.

[43] M. Hammer and D. McLeod, "Database Description with SDM: A Semantic Database Model," *ACM Transactions on Database Systems*, 1981.

[44] SeatGuru, "Airline Seat Classes Explained," https://www.seatguru.com/.

[45] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, 2009.

[46] X. Luo, "Electronic Payment Systems," *Springer*, 2011.

[47] W. Kent, *Data and Reality*. North-Holland, 1978.

[48] E. F. Codd, "Further Normalization of the Data Base Relational Model," *Courant Computer Science Symposia*, 1972.